Project - High Level Design

on

Real Estate Support Triage Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Suhani Pathak	EN22CS301990
2.	Surbhi Jadam	EN22CS3011002
3.	Vanshika Soni	EN22CS3011061
4.	Vidhi Yadav	EN22CS3011077
5.	Vinay	EN22CS3011082

Group Name: Group07D9

Project Number: AAI-07

Industry Mentor Name: Ashwin Krishnan CK

University Mentor Name: Prof. Pradeep Baniya

Academic Year: 2026

Table of Contents

1. Introduction

The Real Estate Support Triage Agent is a production-grade, AI-powered automation system designed to streamline the initial response workflow for incoming tenant and property management communications. Built using Google Gemini 2.5 Flash via the LangChain framework, the system operates as a fully reactive multi-step pipeline that processes unstructured natural language messages end-to-end — from raw text input to a professionally drafted reply — without requiring human intervention at any intermediate stage.

This document describes the complete system design of the triage agent, including its architecture, components, data flows, API catalogue, data model, non-functional characteristics, and security considerations. It serves as a primary reference for development, integration, testing, and future maintenance.

1.1 Scope of the Document

This document covers the detailed technical design of the Real Estate Support Triage Agent system. It includes:

- Internal architecture and component-level design
- Process and information flow
- API catalogue and interface design
- UML diagrams and system structure
- LLM selection rationale
- Approach toward solving the problem statement

This document does not cover deployment infrastructure, CI/CD pipelines, or future enhancements.

1.2 Intended Audience

Audience	Purpose
Developers	Understand component design and implementation
Evaluators / Faculty	Assess technical depth and design quality

Testers	Design test cases based on system flow
Project Team	Reference for development and maintenance

1.3 System Overview

The Real Estate Support Triage Agent is a multi-stage AI pipeline system designed to automate tenant communication handling.

The system processes incoming messages through three intelligent stages:

1. **Classification Agent**
   Determines urgency (HIGH / MEDIUM / LOW) and intent (maintenance, inquiry, complaint, payment, viewing)
2. **NER Extraction Agent**
   Extracts structured information such as:
   - Tenant ID
   - Unit number
   - Property address
   - Issue type
   - Dates
3. **Response Generation Agent**
   Generates a professional, context-aware draft reply based on extracted data and urgency level

The system is exposed through a FastAPI-based REST API and operates entirely using Google Gemini 2.5 Flash, ensuring fast and cost-efficient execution.

2. System Design

2.1 Application Design

The application follows a modular layered architecture:

- **API Layer — api/server.py**
  - Handles HTTP requests
  - Validates input using Pydantic
  - Returns structured JSON responses
- **Orchestration Layer — main.py**

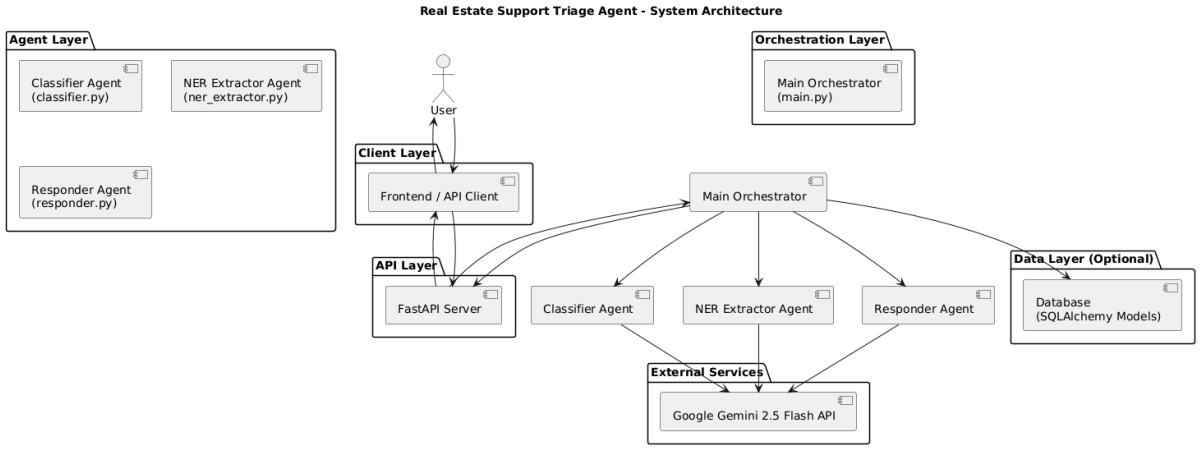
- Controls execution of all agents
- Calls classifier → NER → responder sequentially

🔹 **Agent Layer — agents/**

- classifier.py → Classifies message
- ner_extractor.py → Extracts entities
- responder.py → Generates response

🔹 **Data Layer — database/models.py**

- SQLAlchemy ORM schema for storing triage data (planned)



2.2 Process Flow

The system operates in the following steps:

Step 1 — Message Ingestion

User sends a message via POST /triage endpoint

Step 2 — Classification

Message is analyzed to determine urgency and intent

Step 3 — Entity Extraction

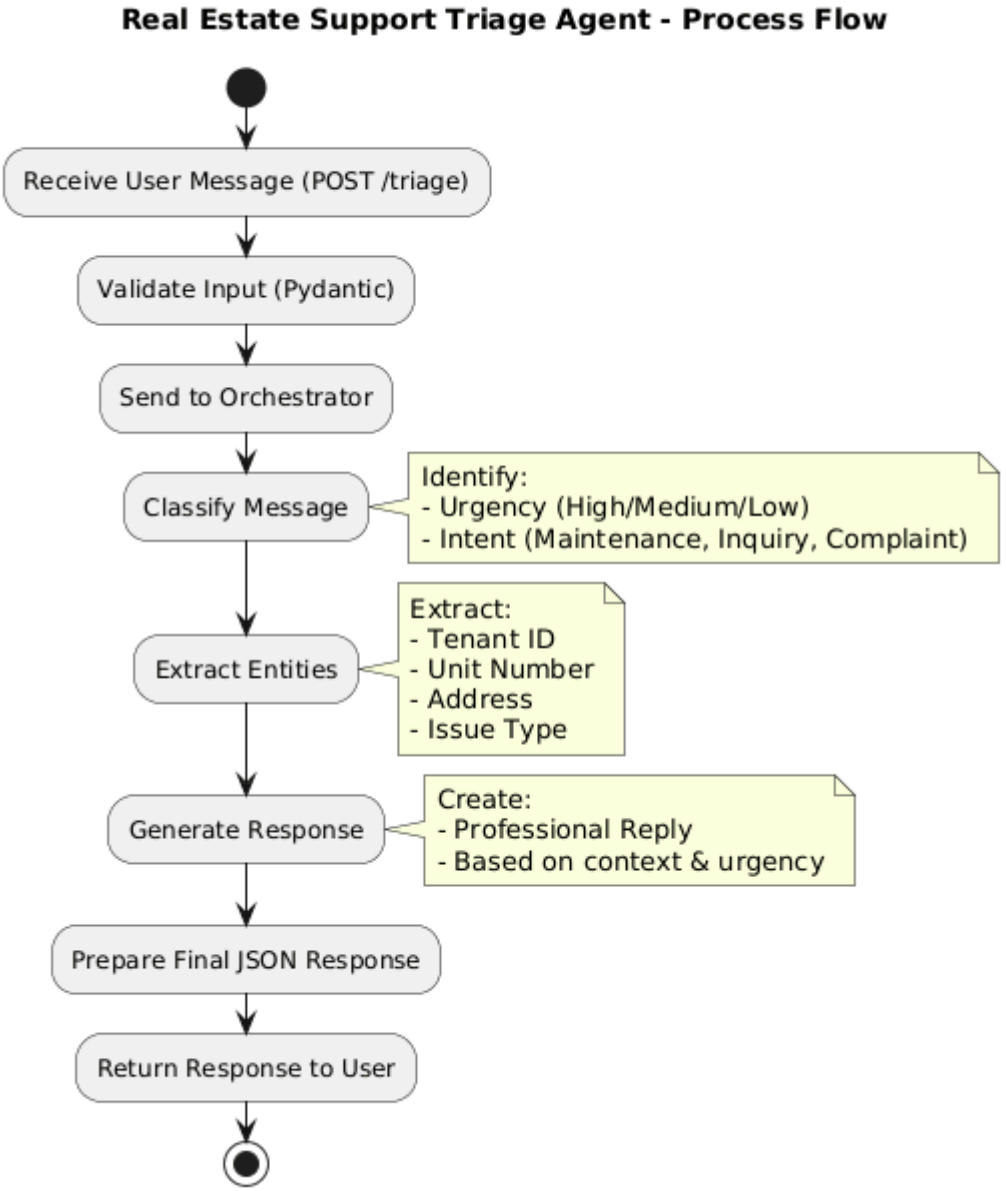
Relevant structured data is extracted from the message

Step 4 — Response Generation

A professional reply is generated using context

Step 5 — Response Output

Final structured JSON response is returned to the user



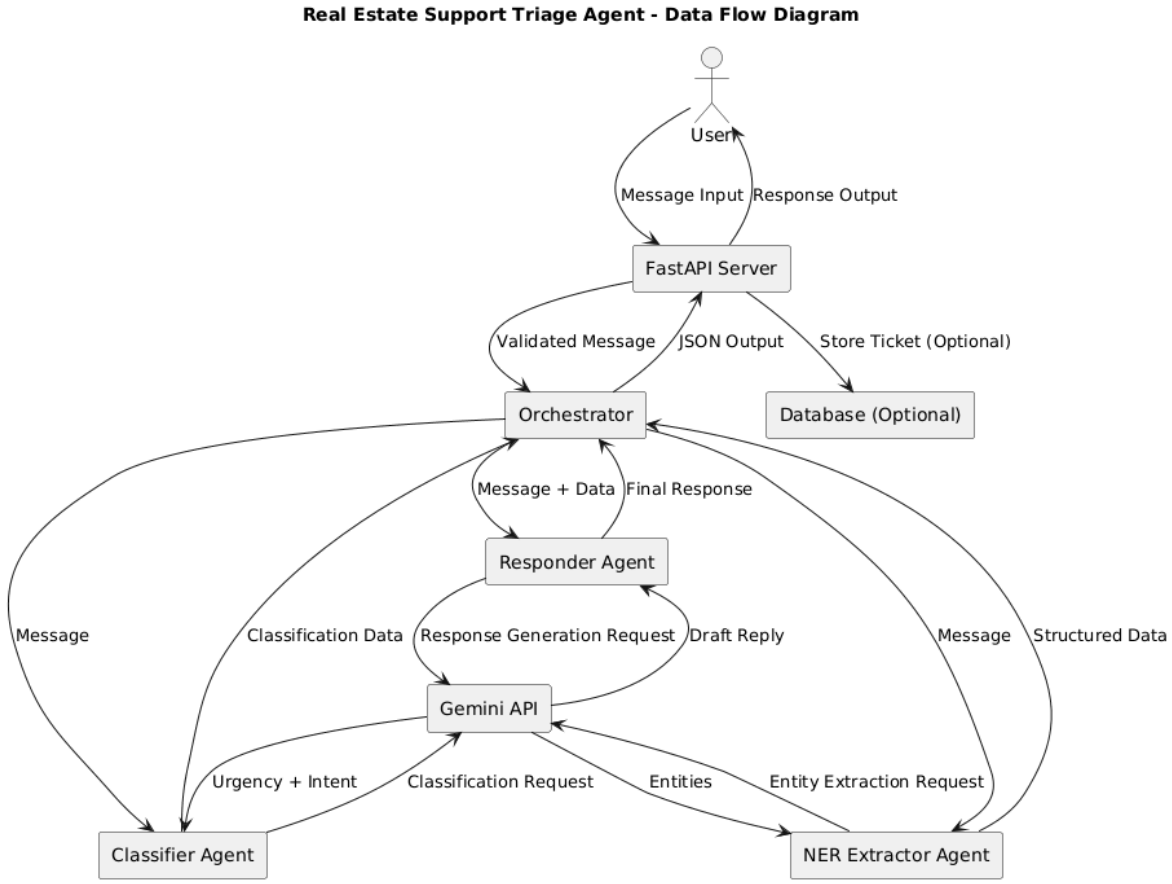
2.3 Information Flow

The system follows a linear transformation pipeline:

Stage	Input	Output
Input	Raw message	Message string

Classification	Message	Urgency + Intent
NER	Message	Structured entities
Response	All inputs	Draft reply
Output	Structured data	Final JSON

Each stage refines the data from unstructured text to structured output.

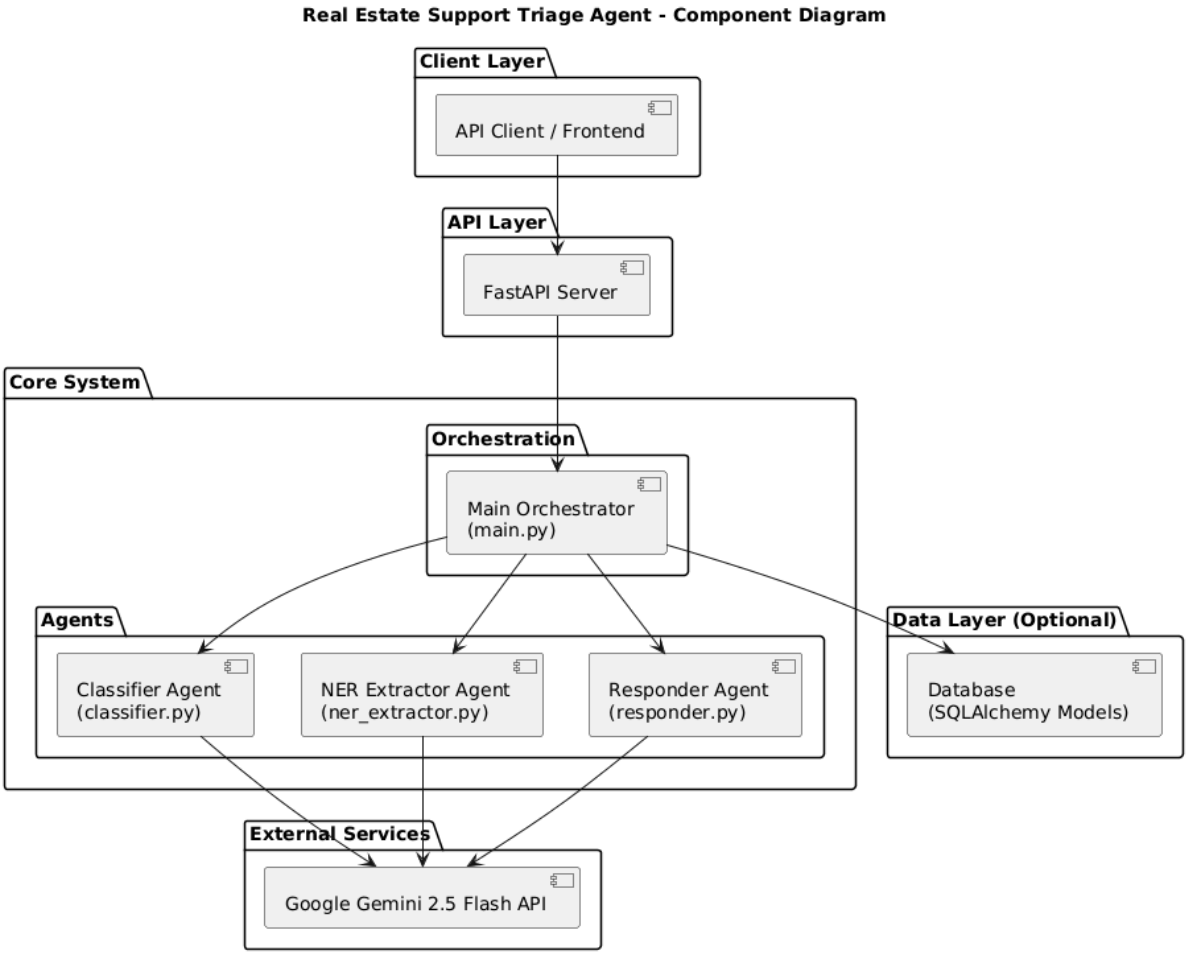


2.4 Components Design

The system consists of the following core components:

- **Classifier Agent**
  Identifies urgency and intent using LLM
- **NER Extractor Agent**
  Extracts structured data fields
- **Responder Agent**
  Generates human-like responses

- **Orchestrator (main.py)**
  Connects all agents in sequence
- **API Server**
  Provides REST endpoints for interaction



2.5 Key Design Considerations

1. **Why the Agentic Pipeline?**
   Breaking the system into multiple agents improves modularity, accuracy, and maintainability.
2. **Why Not RAG?**
   The system focuses on real-time message processing rather than knowledge retrieval, making agent-based processing more suitable.
3. **Why Structured JSON Output?**
   Ensures consistent API responses and easy integration with frontend systems.
4. **Why Different Temperatures?**
   - Classification & NER → temperature = 0 (deterministic)
   - Response generation → temperature = 0.3 (natural replies)

5. **Why FastAPI?**
   Provides high performance, automatic validation, and API documentation.
6. **Why Google Gemini 2.5 Flash?**
   - Fast response time
   - Good structured output
   - Free tier availability
   - Strong zero-shot capabilities

2.6 API Catalogue

Endpoint	Method	Purpose
/	GET	System status
/health	GET	Health check
/triage	POST	Main processing endpoint
/docs	GET	Swagger UI

3. Data Design

3.1 Data Model

The system defines a **Triage Ticket object**:

Field	Description
original_message	Raw input message
urgency	HIGH / MEDIUM / LOW
intent	Message category
entities	Extracted structured data
draft_reply	Generated response
source	Input source
timestamp	Record creation time

3.2 Data Access Mechanism

- In-memory processing during request lifecycle
- ORM-based persistence (planned)
- Function-based data flow between modules

3.3 Data Retention Policies

- Active tickets stored until resolved
- Historical data retained for analysis
- Sensitive data handled securely
- Future automated cleanup planned

3.4 Data Migration

Future migration strategies include:

- SQLite → PostgreSQL
- Local → Cloud deployment
- ORM-based schema migrations using Alembic

4. Interfaces

External Interface

- Google Gemini API

Internal Interfaces

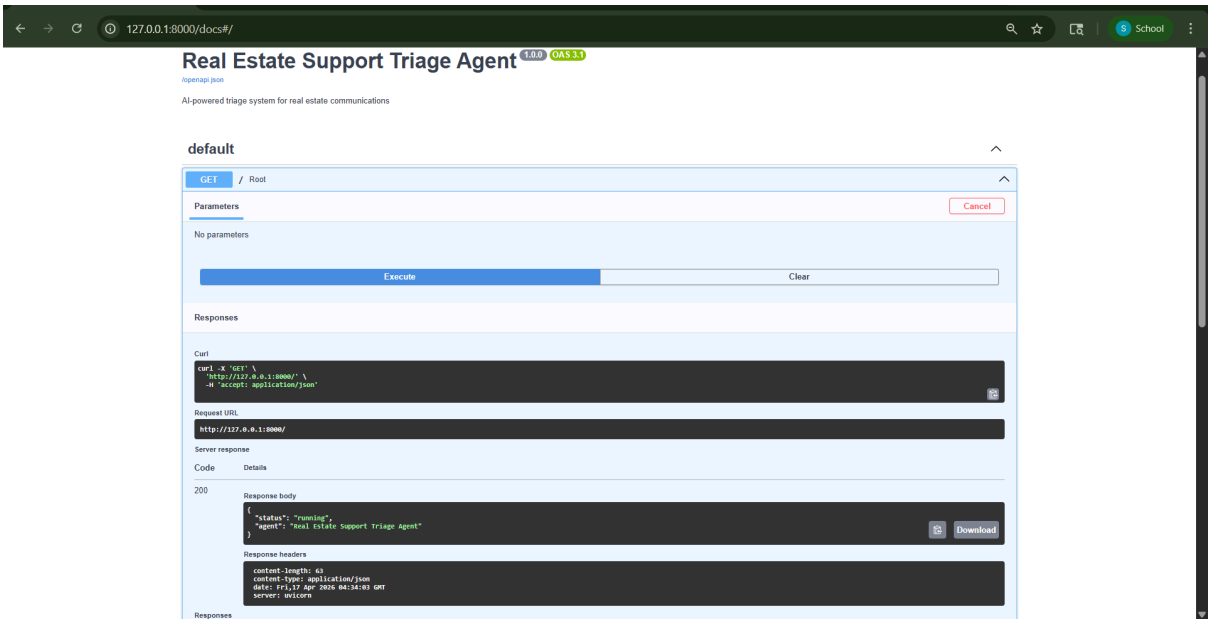
- Function calls between agents

User Interface

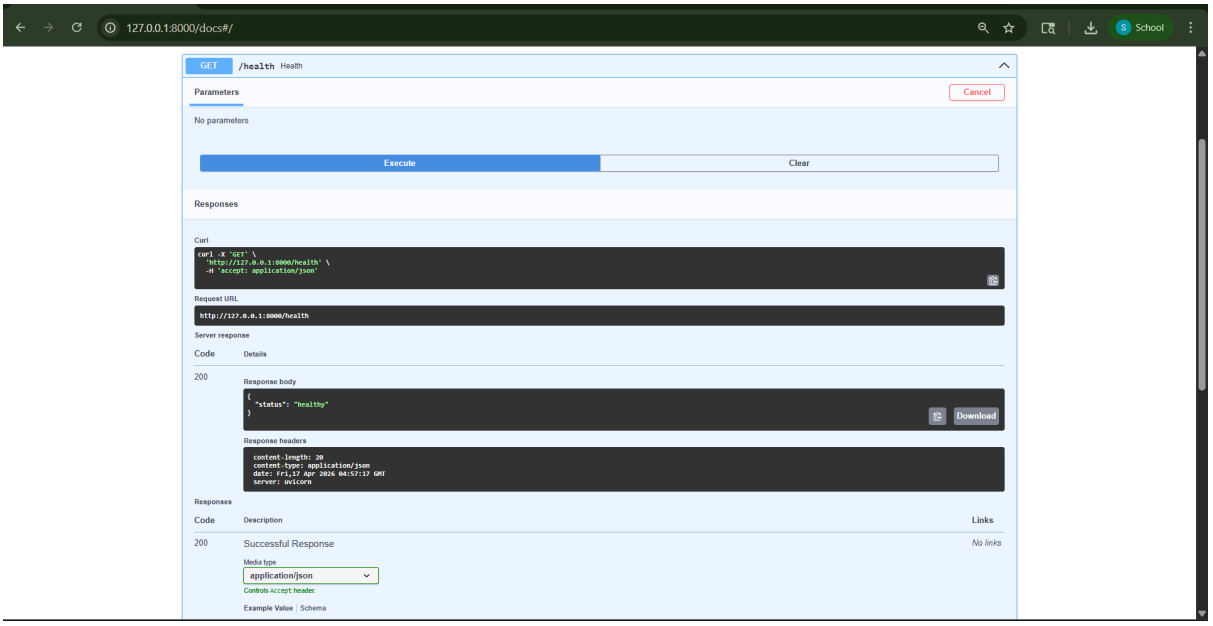
- REST API endpoints

Screenshots of the Project:

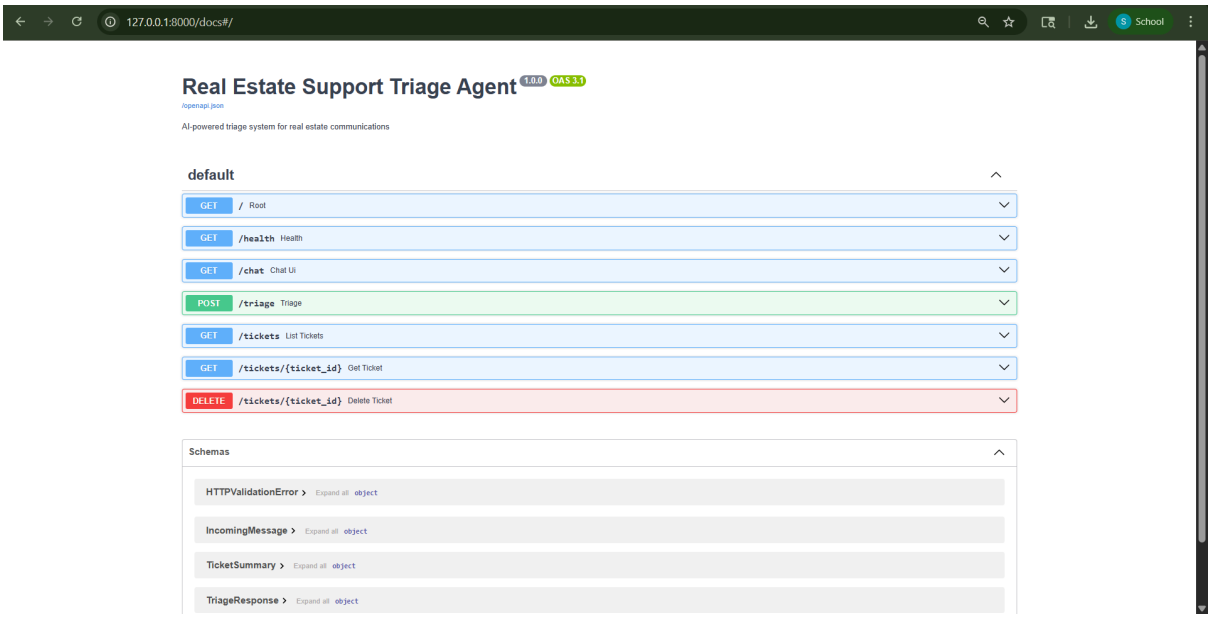
1. Home / Root Endpoint



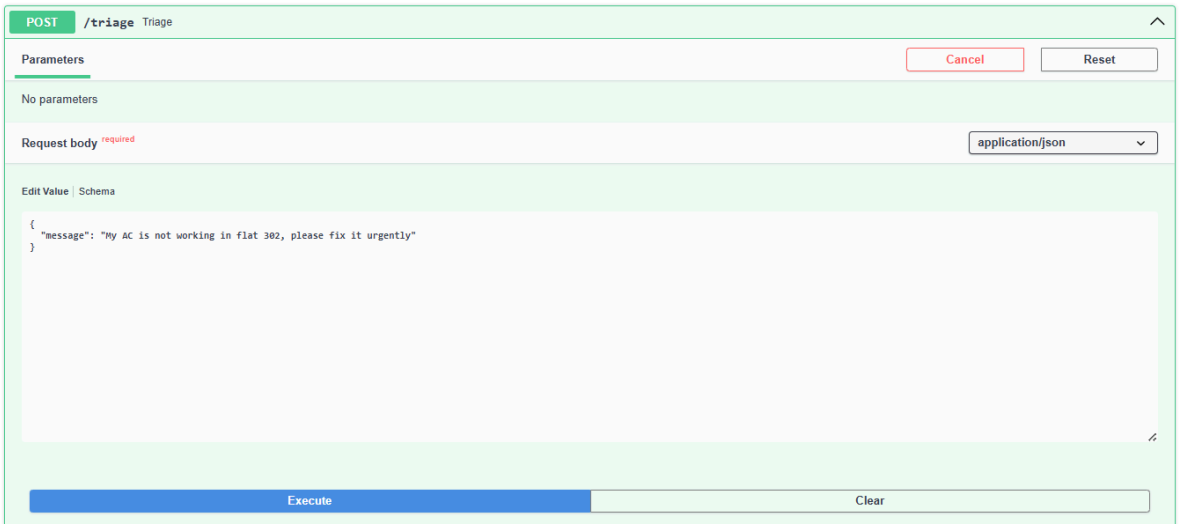
2. Health Check Endpoint



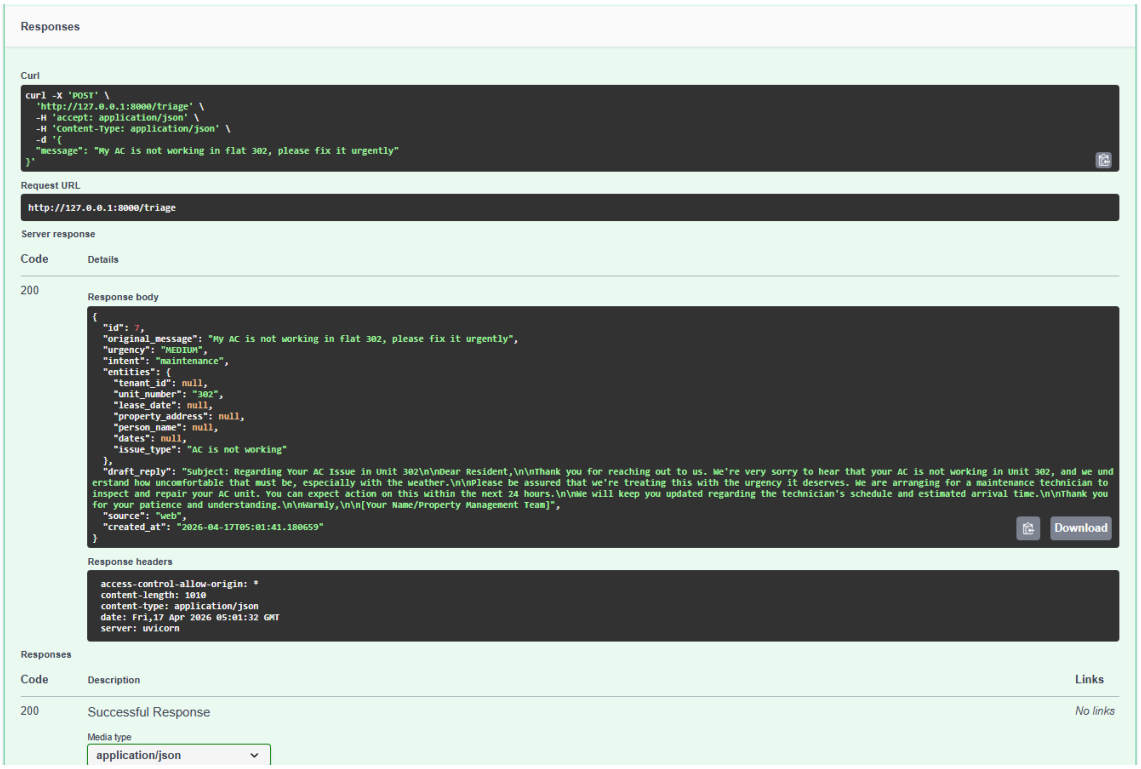
3. Swagger UI (API Documentation Page)



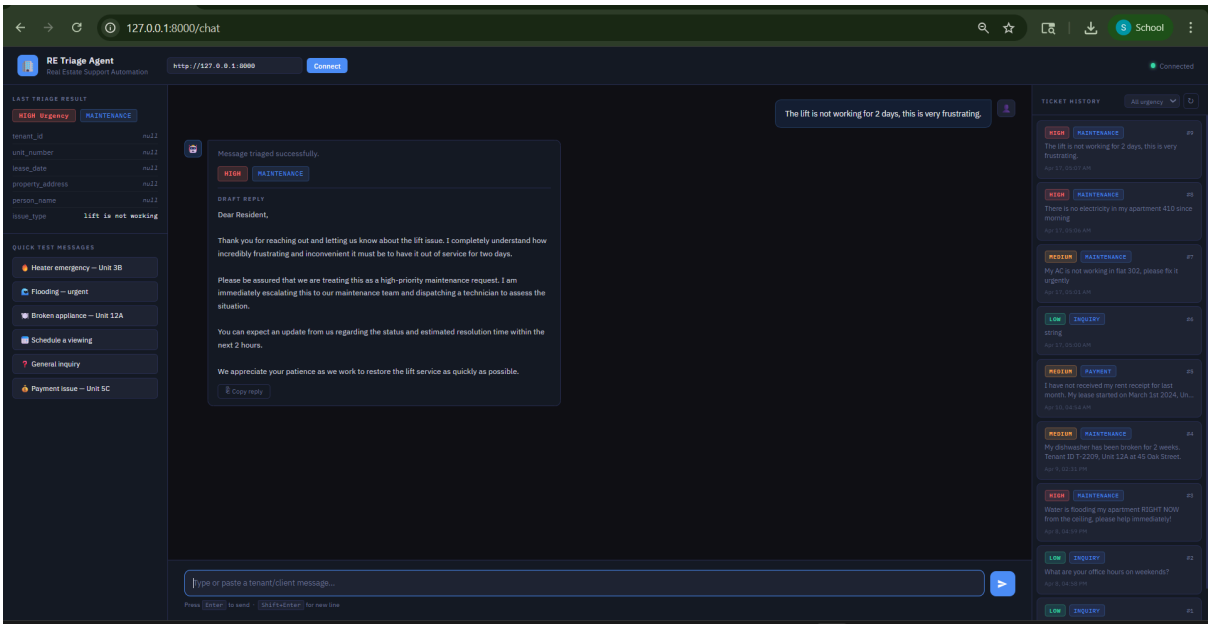
4. Triage API Request (Input)



5. Triage API Response (Output)



6. Chat Interface / Frontend UI Screenshot



5. State and Session Management

The system is **stateless**:

- Each request is independent
- No session data stored
- Supports scalability and distributed deployment

6. Caching

Currently not implemented

Future improvements:

- Response caching
- Classification caching

- API result caching

7. Non-Functional Requirements

7.1 Security Aspects

- API keys stored in *.env*
- Input validation using Pydantic
- Protection against malformed inputs
- Future authentication mechanisms

7.2 Performance Aspects

Current:

- Response time: ~5–15 seconds

Optimizations:

- Parallel execution (async)
- Reduced API latency
- Scalable backend deployment

8. References

Reference	URL
Google Gemini API Documentation	https://ai.google.dev/gemini-api/docs
Google GenAI Python SDK	https://github.com/googleapis/python-genai

FastAPI Documentation	https://fastapi.tiangolo.com
LangChain Documentation	https://python.langchain.com
SQLAlchemy Documentation	https://docs.sqlalchemy.org
Pydantic Documentation	https://docs.pydantic.dev
Uvicorn ASGI Server Documentation	https://www.uvicorn.org
PlantUML Documentation	https://plantuml.com